Dedeepya Majety: Accenture Technical & Architectural Interview Dossier

Target: Accenture in India — Data Engineer (Requisition ID: 4476918574)

Candidate: Dedeepya Majety
Current Role: Associate - Data Engineer @ Cognizant (3.9 YOE)
Target Level: Data Engineer / Senior Analyst (Data & AI Platform Engineering)
Location: Bengaluru, Karnataka, India (BDC14A - STPI)
Notice Period Status: 🟢 Serving Official Notice Period · Available in 30 Days
Compensation Floor Anchor: Confirmed Offer @ 10.9 LPA · **Target Range:** 16.0 – 21.0+ LPA
Verified Credentials: Databricks Certified Data Engineer Associate (Badge ID: 185421754), Microsoft Certified PL-300, Microsoft Certified PL-900

---

1. Executive Positioning & Recruiter/HM Opening Hook

1.1 The 60-Second Verbal Opening Hook (Memorize & Deliver Naturally)

> *"I am an Azure Data Engineer with nearly 4 years of experience at Cognizant, specializing in building and scaling distributed data platforms on Azure Databricks, PySpark, and Delta Lake.*
>
> *In my current engagement, I led the migration of 40+ legacy Informatica ETL workflows to an Azure Databricks Medallion Lakehouse. By optimizing PySpark vectorized executions, eliminating shuffle bottlenecks, and using Auto Loader for incremental ingestion, our team slashed average pipeline execution times by 40% and reduced cloud infrastructure compute spend by ~30%, while maintaining 99.8% pipeline SLA uptime through automated Apache Airflow orchestration.*
>
> *Because Accenture's JD explicitly requires a minimum of 3 years of hands-on experience in Microsoft Azure Databricks, my 3.9 years of direct enterprise delivery combined with my Databricks Certified Data Engineer Associate certification allows me to step in as a productive contributor from day one. I am currently serving my official notice period with an offer in hand at ~11 LPA and am available to join Accenture within 30 days."*

---

2. Accenture Practice Context & Alignment

Accenture Practice Dimension	Accenture Requirement	Dedeepya Production Proof Point (Cognizant)
Databricks Mandate	Minimum 3 years Microsoft Azure Databricks experience	3.9 YOE direct production delivery on Azure Databricks clusters, Delta Lake, Spark SQL, and PySpark; certified Associate Data Engineer.
Pipeline Scale & ETL	Cloud First data pipeline modernization and optimization	Modernized 40+ Informatica PowerCenter workflows to PySpark + Delta Lake, cutting batch runtimes by 40%.
Orchestration & SLAs	Automated workflow scheduling and SME support	Authored modular Python Airflow DAGs with automatic retries and SLA alerting, sustaining 99.8% pipeline availability.
Data Quality & Governance	Data validation, reconciliation, and auditability	Row-count and checksum parity verification, Unity Catalog fine-grained access control, and Informatica MDM deduplication.

---

3. Top 10 Technical Viva & Architecture Defense Questions for Accenture

Q1: Accenture focuses heavily on large-scale PySpark performance. How do you troubleshoot and fix data skew in a Spark job?

Structured Viva Answer: - **How to Identify Skew:** Look at the Spark UI: - In the Stages tab, check the task timeline and summary metrics. If the maximum task duration is 10x to 50x greater than the median task duration, or if 199 tasks finish in seconds while 1 task takes 45 minutes, you have data skew. - Check the "Shuffle Read Size / Records" and "Spill (Memory)" / "Spill (Disk)" columns. A skewed partition will show massive shuffle read and large spill values. - **Root Cause:** A join key or grouping key has a disproportionately high frequency (e.g., `NULL`, default placeholder `UNKNOWN`, or a massive corporate entity ID). - **Remediation Techniques:** 1. **Broadcast Hash Join (BHJ):** If one DataFrame is smaller than 10MB (or up to a few hundred MB if executor RAM permits), force a broadcast join: `df1.join(F.broadcast(df2), "key")`. This completely eliminates the shuffle stage. 2. **Salting the Skewed Key:** - For the large skewed DataFrame, append a random integer salt from 0 to N-1: `df_skewed.withColumn("salted_key", F.concat(F.col("key"), F.lit("_"), F.floor(F.rand() * 10)))`. - For the lookup DataFrame, explode it 10 times with salts 0 to 9. - Join on `salted_key`. This distributes the hot key across 10 separate executor partitions evenly. 3. **Adaptive Query Execution (AQE):** Ensure `spark.sql.adaptive.enabled = true` and `spark.sql.adaptive.skewJoin.enabled = true`. Spark detects skewed partitions at runtime and splits them into smaller sub-partitions automatically.

Q2: How does Spark manage executor memory, and what causes an OutOfMemoryError (OOM)?

Structured Viva Answer: - **Spark Unified Memory Manager:** Divides the JVM heap into two primary regions: 1. **Execution Memory (Shuffle/Joins/Aggregations):** Used for shuffles, hash joins, sorting, and aggregation buffers. 2. **Storage Memory (Caching/Broadcast):** Used for cached DataFrames (`.persist()`, `.cache()`) and broadcast variables. - **Dynamic Borrowing:** Execution memory can evict cached data from Storage memory if needed. Storage cannot evict Execution memory. - **User Memory (40% default):** Holds user-defined data structures, internal metadata, and Python UDF objects. - **Causes of OOM:** - **Driver OOM (`java.lang.OutOfMemoryError: Java heap space`):** Calling `.collect()` or `.toPandas()` on a large dataset; broadcasting a table larger than driver RAM; downloading huge logs. - **Executor OOM (`Container killed by YARN/K8s for exceeding memory limits`):** - Large shuffle partitions spilling and exhausting container memory. - Python worker memory overhead: heavy Python UDFs deserializing millions of rows from JVM into Python runtime. - Skewed join partition that cannot fit into an executor's execution memory buffer. - **Cognizant Production Fix:** Replaced row-by-row Python UDFs with native PySpark functions (`pyspark.sql.functions`), increased `spark.sql.shuffle.partitions` from default 200 to match data volume, and tuned `spark.executor.memoryOverhead`.

Q3: How did you achieve a 40% runtime reduction and ~30% cost savings during the Informatica to Databricks migration?

Structured Viva Answer: - **Runtime Reduction (40%):** 1. **Vectorized PySpark vs Row-by-Row ETL:** Replaced legacy procedural cursors and row-level lookups with native vectorized Catalyst physical plans. 2. **Auto Loader Ingestion:** Switched from batch polling to Databricks Auto Loader (`cloudFiles`), reading only incremental delta files instead of scanning entire directories. 3. **Partition Pruning & File Sizing:** Compacted small files using Delta `OPTIMIZE` and applied Z-Ordering on primary filter keys, reducing Parquet scan I/O by over 60%. 4. **Broadcast Joins:** Converted large shuffle joins with dimension tables into zero-shuffle broadcast joins. - **Cost Reduction (~30%):** 1. **Single-Node / Auto-Scaling Clusters:** Configured aggressive cluster auto-termination (15 minutes idle timeout) and auto-scaling worker nodes (2 to 8 nodes) instead of static 24/7 clusters. 2. **Job Compute vs All-Purpose Clusters:** Executed production Airflow pipelines strictly on automated Databricks Job Clusters (charged at ~0.15 DBU/hr) rather than expensive All-Purpose Interactive Clusters (charged at ~0.40+ DBU/hr). 3. **Spot Instances:** Leveraged Azure Spot VMs for worker nodes on non-critical backfill runs with automatic driver fallback.

Q4: Explain the Delta Lake Transaction Log (`_delta_log`) and how it guarantees ACID transactions.

Structured Viva Answer: - **The Protocol:** Every table write (append, merge, delete, update) generates an atomic JSON commit file in the `_delta_log/` directory: `000000.json`, `000001.json`, etc. - **Commit Contents:** Each JSON file lists exact actions: - `add`: Path to newly written Parquet files with min/max statistics. - `remove`: Path to logical files deleted or replaced (files are not physically removed until `VACUUM`). - `commitInfo`: Timestamp, operation name, user, notebook/job ID. - **Checkpoints (Parquet):** Every 10 commits, Delta writes a `.checkpoint.parquet` file consolidating state. A reader only reads the latest checkpoint and subsequent JSON logs, avoiding reading thousands of historical commit files. - **Optimistic Concurrency Control (OCC):** - Delta assumes transactions will not conflict. - When two transactions commit simultaneously, Delta checks if the files read by Transaction 2 were altered by Transaction 1. - If no mutual file conflict exists, both commit successfully. If a conflict occurs, Transaction 2 throws a `ConcurrentModificationException` or automatically retries.

Q5: What is the difference between `repartition()` and `coalesce()` in PySpark?

Structured Viva Answer: - **`repartition(n)`:** - Full shuffle across all cluster nodes. - Can increase or decrease the number of partitions. - Balances data uniformly across all partitions, eliminating skew. - Expensive network I/O cost. - **`coalesce(n)`:** - Avoids full shuffle by collapsing neighboring partitions on the same executor. - Can **only decrease** the number of partitions (never increase). - Computationally efficient; ideal right before writing data to disk to avoid creating thousands of small Parquet files. - Caveat: If reducing drastically (e.g., from 1000 to 2), can cause upstream operations to run on fewer executors, creating bottlenecks.

Q6: How do you design an enterprise Medallion Lakehouse pipeline from raw landing to BI consumption?

Structured Viva Answer: - **Bronze (Ingestion):** - Streaming ingestion via Auto Loader from ADLS Gen2 (`cloudFiles`). - Stores raw files as append-only Parquet with schema inference and `_rescued_data`. - **Silver (Cleansing & Standardization):** - Cleansed data: data types cast, null handling, deduplication using window functions. - Merged using Delta ACID `MERGE INTO` (Upsert logic). - **Gold (Curated Star Schema):** - Conformed Facts and Dimensions optimized with Liquid Clustering or Z-Ordering on primary join/filter keys. - Consumed by Databricks SQL Warehouse and Power BI.

Q7: What happens when you run `VACUUM` on a Delta table, and what precautions must you take?

Structured Viva Answer: - **What it does:** `VACUUM table_name [RETAIN num HOURS]` physically deletes underlying Parquet files that were logically marked as `remove` in the transaction log older than the retention threshold. - **Default Retention:** Default is 7 days (168 hours). - **Precautions:** - Running `VACUUM` permanently destroys Time Travel capability for versions older than the retention period. - Setting retention to 0 hours requires disabling the safety check (`spark.databricks.delta.vacuum.parallelDelete.enabled` and `delta.retentionDurationCheck.enabled = false`). If concurrent writers or long-running queries are reading older snapshots, setting retention < 7 days can corrupt active reader queries with `FileNotFoundException`.

Q8: How do you handle schema evolution when a source API changes unexpectedly?

Structured Viva Answer: - **Auto Loader Rescue:** With `.option("cloudFiles.schemaLocation", "/checkpoint/schema")`, Auto Loader catches newly added fields and places unparseable values into a special column named `_rescued_data`. - **Merge Schema on Write:** `.writeStream.option("mergeSchema", "true")` automatically updates table metadata without dropping historical partitions. - **Monitoring & Alerts:** Authored pipeline checks that inspect the schema dictionary after read. If new unmapped fields appear, an Airflow webhook notification alerts the engineering team while processing the valid columns safely.

Q9: How do you architect Apache Airflow DAGs for idempotency and disaster recovery?

Structured Viva Answer: - **Idempotency Invariant:** A pipeline must produce identical state whether run once or rerun five times for the same logical date (`ds`). - **Implementation:** - Never use naive `APPEND` writes in scheduled batch pipelines; use partition overwrites (`replaceWhere = "date = '2026-10-12'"`) or Delta `MERGE INTO`. - Pass

logical execution dates (`{{ ds }}`) as pipeline parameters, not system `current_date()` . - Modular task boundaries: `Ingest_Bronze >> Validate_Bronze >> Conform_Silver >> Optimize_Gold` . - Automated retries with exponential backoff (`retries=3` , `retry_delay=timedelta(minutes=5)`).

Q10: How do you optimize Databricks clusters for cost and throughput?

Structured Viva Answer: - **Job Clusters over All-Purpose Clusters:** Always schedule production workloads on automated Job Clusters. - **Cluster Policies:** Enforce max worker nodes, required tags (Environment, Project, CostCenter), and auto-termination timeouts (15 minutes). - **Photon Engine:** Enable Photon acceleration on I/O-heavy SQL, join, and aggregation workloads for 2x to 4x faster execution, netting lower total DBU consumption. - **Storage Tiering:** ADLS Gen2 lifecycle policies to transition raw Bronze data to Cool/Archive tiers after 90 days.

---

4. Live Coding Defense Drills (PySpark)

Drill 1: Identifying and Filtering Corrupt Records with PySpark

```
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

# Define strict target schema
schema = StructType([
    StructField("transaction_id", StringType(), False),
    StructField("customer_id", StringType(), False),
    StructField("amount", DoubleType(), True),
    StructField("status", StringType(), True)
])

# Read with permissive mode and rescued data column
raw_df = (
    spark.read
    .option("mode", "PERMISSIVE")
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .schema(schema)
    .json("abfss://landing@dwhdatalake.dfs.core.windows.net/transactions/")
)

# Split clean records from corrupt records
clean_df = raw_df.filter(F.col("_corrupt_record").isNull()).drop("_corrupt_record")
corrupt_df = raw_df.filter(F.col("_corrupt_record").isNotNull())
```

Drill 2: Salting Technique to Resolve Skewed Joins

```
import pyspark.sql.functions as F

SALT_ROUNDS = 8

# Large skewed DataFrame: add random salt
df_skewed_salted = large_df.withColumn(
    "salt", F.floor(F.rand() * SALT_ROUNDS)
).withColumn(
    "salted_join_key", F.concat(F.col("join_key"), F.lit("_"), F.col("salt"))
)

# Small/Lookup DataFrame: replicate across all salts via explode
lookup_exploded = small_df.withColumn(
    "salt_array", F.array([F.lit(i) for i in range(SALT_ROUNDS)])
).withColumn(
    "salt", F.explode("salt_array")
).withColumn(
    "salted_join_key", F.concat(F.col("join_key"), F.lit("_"), F.col("salt"))
)

# Execute balanced join without partition bottlenecks
result_df = df_skewed_salted.join(
    lookup_exploded,
    on="salted_join_key",
    how="inner"
).drop("salt", "salt_array", "salted_join_key")
```

---

5. Reverse Inversion Questions to Ask Accenture Interviewers

1. *"How are Accenture's delivery teams balancing Databricks Asset Bundles (DABs) versus standard Azure DevOps / GitHub Actions for CI/CD across enterprise clients?"*
2. *"Are client architectures moving toward unified multi-workspace Unity Catalog deployments, or are many still in transition from legacy Hive metastores?"*
3. *"What is the typical cluster governance model on client accounts: centralized IT platform ownership or decentralized team-level cost centers?"*
4. *"What is the primary technical roadblock currently encountered on your cloud modernization engagements?"*

---

6. Compensation & Notice Period Strategy for Accenture

- **Current Status:** Officially serving 30-day notice period at Cognizant.
- **Anchor Offer:** 10.9 LPA (Infosys confirmed).
- **Target Accenture Compensation:** 16.0 – 21.0 LPA.
- **Script:** *"I am currently serving my official notice period at Cognizant and can join Accenture within 30 days. I have already secured an offer at ~11 LPA. Considering Accenture's minimum 3-year Azure Databricks mandate and my 3.9 years of direct enterprise delivery with certified credentials, I am targeting between 17 and 19 LPA."*